



Trabajo Práctico N°1: Muestreo de señales

Profesores:

Parlanti Gustavo Fernand (Presidente)

Molina German Alcides (Vocal)

Rossi Robreto Raul (Vocal)

Alumnos:

Campos

Testa

Verstraete

Mayo de 2025

Resumen

Realizar un programa aplicativo que sea capaz de digitalizar una señal a través del modulo del ADC disponible en la placa de desarrollo a distintas velocidades de muestreo, las requeridas son 8K/S, 16K/S, 22K/S, 44K/S y 48K/S. Los cambios de las velocidades de muestreo serán realizados con una de las teclas de la placa de evaluación, en forma de un buffer circular. Cada velocidad de muestreo se indicará a través de un color RGB del LED. Con otra tecla de la placa se habilitará la adquisición o se parará la misma (Run/Stop). Los valores adquiridos serán almacenados en memoria en un buffer circular de 512 muestras del tipo q15 (fraccional 15bits) y a su vez serán enviados a través del DAC (de 12bits).

Índice

1. Introducción	3
1.1. Objetivos del proyecto	3
1.2. Aspectos relevantes a tener en cuenta	3
1.2.1. Uso de DMA (Direct Memory Access)	3
1.2.2. Técnica de Doble Buffering (Ping-Pong Buffer)	4
1.2.3. Medición del Tiempo de Procesamiento con el Módulo DWT	4
2. Marco teórico	4
2.1. Señales continuas y discretas	4
2.2. Teorema de Nyquist-Shannon	5
2.3. Reconstrucción de señales	5
2.4. Procesamiento Digital en Sistemas Embebidos	5
2.5. CMSIS-DSP	6
3. Placa de Desarrollo y Herramientas Utilizadas	6
3.1. Microcontrolador STM32F446RE	7
3.2. Núcleo ARM Cortex-M4	7
3.3. Periféricos clave utilizados	7
3.3.1. ADC (Analog-to-Digital Converter)	7
3.3.2. DAC (Digital-to-Analog Converter)	8
3.3.3. DMA (Direct Memory Access)	8
3.3.4. TIM (Timers)	9
3.3.5. GPIO – General Purpose Input/Output	9
3.3.6. Herramientas de desarrollo	9
4. Diseño del sistema	10
4.1. Descripción detallada del funcionamiento	10
4.2. Diagrama de bloques del programa aplicativo	11
5. Programa principal	12
5.1. Implementación del main.c	12
5.2. Configuración inicial de los periféricos	16
5.2.1. Configuración del ADC	16
5.2.2. Configuración del DAC	17
5.2.3. Configuración del GPIO	17
5.2.4. Configuración del DMA	18
5.2.5. Configuración del TIM2	18
6. Resultados experimentales	19
6.1. Pruebas realizadas con diferentes velocidades de muestreo	19
6.2. Análisis de los resultados obtenidos	22
7. Conclusiones	22
8. Bibliografía	23

1. Introducción

En los sistemas embebidos modernos, la digitalización de señales analógicas en tiempo real representa una funcionalidad crítica en aplicaciones como adquisición de datos, procesamiento digital de audio, control industrial y sistemas de comunicación. Este trabajo práctico tiene como objetivo principal el desarrollo de un sistema aplicativo que implemente un flujo completo de adquisición, procesamiento y reproducción de señales analógicas en una placa de desarrollo basada en un microcontrolador STM32F446RE.

El sistema propuesto permite capturar una señal analógica utilizando el módulo ADC (Convertidor Analógico-Digital) del microcontrolador, convertir los valores a formato Q15 (15 bits fraccional en punto fijo), y almacenarlos en un buffer circular de 512 muestras. Posteriormente, estas son reproducidas en tiempo real mediante el módulo DAC (Convertidor Digital-Analógico) integrado en la misma placa. Todo el proceso se realiza en tiempo real y bajo distintas velocidades de muestreo, seleccionables dinámicamente mediante una interrupción de GPIO.

1.1. Objetivos del proyecto

- Implementar la digitalización de señales en tiempo real utilizando el ADC de la placa STM32F446RE.
- Diseñar e integrar un buffer circular para almacenar muestras en formato Q15, asegurando un flujo continuo de datos sin pérdidas.
- Configurar y utilizar el DAC para la reproducción de la señal adquirida.
- Permitir el cambio dinámico de la frecuencia de muestreo (8 kHz, 16 kHz, 22 kHz, 44 kHz y 48 kHz) mediante una tecla de usuario, utilizando una estructura de tipo circular.
- Utilizar un LED RGB como indicador visual de la frecuencia de muestreo seleccionada.
- Implementar una segunda tecla que permita habilitar o detener el proceso de adquisición y reproducción (modo Run/Stop).
- Optimizar el sistema para que funcione de manera eficiente utilizando DMA (Acceso Directo a Memoria) y temporizadores (TIM) del microcontrolador.

1.2. Aspectos relevantes a tener en cuenta

Para lograr una digitalización y reproducción de señales eficiente y en tiempo real, se implementaron varias técnicas clave orientadas a minimizar la latencia, reducir la carga del procesador y mejorar la capacidad de respuesta del sistema. A continuación se describen los principales enfoques adoptados:

1.2.1. Uso de DMA (Direct Memory Access)

El DMA permite que los datos capturados por el ADC y enviados al DAC sean transferidos directamente entre periféricos y memoria sin intervención del CPU. Esta técnica es esencial para evitar interrupciones por cada muestra y reducir drásticamente la latencia en la adquisición y reproducción de señales, permitiendo que el procesador se enfoque exclusivamente en tareas de control o procesamiento si es necesario.

- En el sistema implementado, tanto el ADC como el DAC trabajan en modo DMA.
- Las transferencias se configuran en modo circular para que una vez completado el buffer, el periférico vuelva automáticamente al inicio.

1.2.2. Técnica de Doble Buffering (Ping-Pong Buffer)

Para asegurar una reproducción continua y evitar glitches o corrupción de datos en la señal de salida, se implementó una técnica de doble buffering. En este esquema, el buffer de 512 muestras se divide en dos mitades:

- Mientras el ADC llena una mitad del buffer, la otra mitad procesa los datos obtenidos, en este caso los transforma a Q15 y los adapta para sacar la señal por el DAC
- Al completarse una transferencia, se realiza el "ping-pong" de roles entre mitades.

Esta técnica permite una transición fluida entre adquisición y reproducción, eliminando tiempos muertos y mejorando la latencia total del sistema.

1.2.3. Medición del Tiempo de Procesamiento con el Módulo DWT

Para evaluar el rendimiento del sistema y optimizar su eficiencia, se utilizó el módulo DWT (Data Watchpoint and Trace Unit) disponible en el microcontrolador STM32F446RE. Este módulo permite acceder al registro CYCCNT, que cuenta ciclos de CPU con alta resolución.

- Se utilizó para medir el tiempo exacto de procesamiento por bloque de datos (por ejemplo, el tiempo entre interrupciones DMA half-complete y complete).
- Permite realizar profiling en tiempo real, identificando cuellos de botella en el flujo de datos y verificando que el sistema cumple con las restricciones temporales para cada frecuencia de muestreo.
- Esta métrica es fundamental para garantizar el correcto funcionamiento incluso a altas tasas de muestreo como 48kHz.

2. Marco teórico

El procesamiento digital de señales (DSP) en sistemas embebidos se basa en principios matemáticos y físicos que permiten representar y manipular señales analógicas en forma digital. Esta sección revisa los conceptos esenciales relacionados al muestreo, la reconstrucción de señales, y su representación numérica, todos fundamentales para el desarrollo de aplicaciones de adquisición y reproducción de señales en tiempo real.

2.1. Señales continuas y discretas

Una señal analógica es una función continua en el tiempo que puede tomar un conjunto infinito de valores. En contraste, una señal discreta en el tiempo se define sólo en instantes específicos (secuenciales), y una señal digital es aquella que además tiene valores cuantificados (por ejemplo, números enteros o punto fijo). Para poder procesar una señal analógica en un sistema digital es necesario muestrearla y cuantizarla:

- Muestreo: tomar valores de la señal a intervalos de tiempo regulares $T_s = \frac{1}{f_s}$
- Cuantización: representar cada muestra con un número finito de bits

2.2. Teorema de Nyquist-Shannon

El Teorema de Muestreo de Nyquist-Shannon establece que una señal analógica de ancho de banda limitado f_{max} puede ser reconstruida perfectamente a partir de sus muestras si se cumple:

$$f_s \geq 2f_{max} \quad (1)$$

Si se muestrea por debajo de este umbral, ocurre aliasing, es decir, componentes de alta frecuencia se pliegan en el espectro y aparecen como señales falsas en bajas frecuencias.

2.3. Reconstrucción de señales

La reconstrucción de una señal analógica a partir de sus muestras digitales implica generar una versión continua en el tiempo que se asemeje lo más posible a la señal original. En teoría, esto se logra mediante interpolación ideal con funciones sinc, lo cual no es factible en sistemas reales debido a su complejidad infinita. En la práctica, se utilizan filtros pasa bajos analógicos a la salida del DAC para suavizar la señal escalonada producida por la conversión digital-analógica. La calidad de esta reconstrucción depende de factores como la frecuencia de muestreo utilizada, la resolución de cuantización, y la respuesta del filtro reconstructivo. En sistemas embebidos, esta etapa es crítica en aplicaciones de audio o instrumentación, donde se requiere fidelidad y baja distorsión.

2.4. Procesamiento Digital en Sistemas Embebidos

El procesamiento digital de señales (DSP) en sistemas embebidos implica trabajar con muestras digitalizadas para realizar operaciones como filtrado, transformación, detección o generación de señales. Dado que los microcontroladores suelen tener recursos limitados (memoria, velocidad, consumo), se emplean técnicas optimizadas como el uso de formato Q15, que representa números fraccionarios en punto fijo de 16 bits, permitiendo una aritmética rápida sin necesidad de FPU. Además, se emplean periféricos como DMA para transferencias de datos sin intervención del CPU, y temporizadores para garantizar una temporización precisa en la adquisición y reproducción. Este enfoque permite realizar procesamiento en tiempo real en aplicaciones como audio digital, monitoreo de sensores o control en lazo cerrado. Una vez digitalizada, la señal puede ser procesada con algoritmos de DSP, como:

- Filtros digitales (FIR, IIR)
- Transformadas rápidas (FFT)
- Compresión o codificación

2.5. CMSIS-DSP

La biblioteca **CMSIS-DSP** (Cortex Microcontroller Software Interface Standard - Digital Signal Processing) es una colección de funciones optimizadas desarrolladas por ARM para facilitar el procesamiento de señales digitales en microcontroladores basados en núcleos Cortex-M. Esta biblioteca forma parte del ecosistema CMSIS y proporciona una API estandarizada que incluye operaciones matemáticas básicas, filtros FIR e IIR, transformadas rápidas de Fourier (FFT), convoluciones, correlaciones, estadísticas y operaciones vectoriales, todas implementadas en código eficiente y adaptado al hardware.

Una de las principales ventajas de CMSIS-DSP es que ofrece rutinas en formato **fixed-point** (Q15 y Q31) y **floating-point**, lo cual permite su uso tanto en microcontroladores sin unidad de punto flotante como en aquellos que sí la poseen (por ejemplo, los basados en Cortex-M4 y Cortex-M7 con FPU).

Estas funciones están altamente optimizadas en ensamblador para los núcleos ARM, aprovechando instrucciones SIMD (Single Instruction, Multiple Data) y otras características de hardware, logrando un alto rendimiento con un bajo consumo de recursos.

3. Placa de Desarrollo y Herramientas Utilizadas

El proyecto fue desarrollado sobre la placa STM32 Nucleo-F446RE, basada en el microcontrolador STM32F446RE, perteneciente a la familia STM32F4 de STMicroelectronics. Esta plataforma integra una arquitectura avanzada ARM Cortex-M4, periféricos orientados a procesamiento de señales, y herramientas de desarrollo que permiten implementar sistemas en tiempo real de forma eficiente.

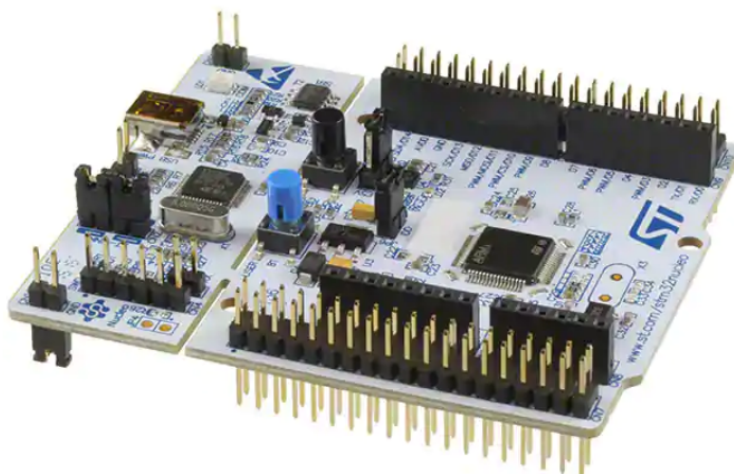


Figura 1: Placa de desarrollo

3.1. Microcontrolador STM32F446RE

El STM32F446RE es un microcontrolador de 32 bits basado en el núcleo ARM Cortex-M4, optimizado para procesamiento digital con soporte de instrucciones DSP y unidad de punto flotante opcional (FPU simple precisión). Sus principales características relevantes al proyecto son:

- Frecuencia de operación: hasta 180MHz
- Memoria Flash: 512KB
- RAM: 128KB
- Tensión de operación: 1.8V a 3.6V
- Paquete: LQFP64 (compatible con la Nucleo-F446RE)

3.2. Núcleo ARM Cortex-M4

El núcleo ARM Cortex-M4 es el corazón del microcontrolador STM32F446RE y está especialmente diseñado para aplicaciones que requieren una combinación de procesamiento eficiente, bajo consumo y capacidad de ejecución de algoritmos DSP (procesamiento digital de señales) en tiempo real. Se trata de una arquitectura de 32 bits basada en el conjunto de instrucciones Thumb-2, que combina instrucciones de 16 y 32 bits para mejorar la eficiencia del código y reducir el uso de memoria, una característica especialmente valiosa en sistemas embebidos con recursos limitados.

Una de las principales ventajas del Cortex-M4 respecto de sus predecesores es su capacidad nativa de ejecutar instrucciones DSP, como multiplicación acumulada (MAC, Multiply-and-Accumulate), operaciones SIMD (Single Instruction Multiple Data), y saturación aritmética, lo cual lo convierte en una excelente opción para aplicaciones de filtrado digital, transformadas rápidas (FFT) y otras tareas típicas de procesamiento de señales. Estas instrucciones están directamente soportadas por el compilador y son aceleradas por hardware, lo que reduce el número de ciclos por operación y mejora considerablemente el rendimiento en tiempo real.

El Cortex-M4 también incluye un sistema de interrupciones NVIC altamente eficiente, que permite la respuesta rápida a eventos críticos como interrupciones del ADC, botones o DMA, lo cual es fundamental en aplicaciones que requieren tiempos deterministas. Además, incorpora el módulo DWT (Data Watchpoint and Trace), que ofrece herramientas de diagnóstico avanzadas como el contador de ciclos de CPU (CYCCNT), ideal para realizar mediciones de tiempo precisas, evaluación de latencia y profiling de código sin necesidad de hardware externo.

Aunque algunos modelos de Cortex-M4 incluyen una unidad de punto flotante (FPU) de precisión simple, en este proyecto se optó por trabajar con formato Q15 (punto fijo) para maximizar la eficiencia y mantener la compatibilidad con la biblioteca CMSIS-DSP, que explota al máximo las capacidades nativas del núcleo para operar en enteros fraccionales.

3.3. Periféricos clave utilizados

3.3.1. ADC (Analog-to-Digital Converter)

El STM32F446RE cuenta con 3 ADCs independientes (ADC1, ADC2 y ADC3), cada uno de 12 bits de resolución y con capacidad de hasta 2.4 MSPS (mega muestras por segundo) en modo intercalado. Para este proyecto se utiliza el ADC1 en modo regular y con DMA activado. Sus características técnicas relevantes incluyen:

- Resolución seleccionable: 6, 8, 10 o 12 bits (se usa 12 bits para mayor precisión).
- Tasa de muestreo máxima (single conversion): hasta 2.4 MSPS.
- Rango de entrada: 0V a VREF (típicamente 3.3V).
- Tiempo de muestreo configurable: desde 3 hasta 480 ciclos de ADC.
- Trigger externo: el inicio de la conversión puede ser disparado por un temporizador, lo cual permite muestrear a frecuencias precisas.
- Modo DMA: transferencia automática de los resultados a un buffer en memoria (sin intervención del CPU).
- Modo continuo + DMA circular: permite muestreo ininterrumpido con mínimo overhead.

3.3.2. DAC (Digital-to-Analog Converter)

El STM32F446RE incluye 2 canales DAC (DAC1 y DAC2), ambos de 12 bits, con capacidad de salida en modo bufferizado (con baja impedancia) o sin buffer (alta velocidad). Se utiliza DAC1 en este proyecto para convertir las muestras digitales de salida nuevamente en una señal analógica. Características clave:

- Resolución de conversión: 12, 10 u 8 bits.
- Velocidad típica: hasta 1 MSPS (modo sin buffer).
- Modo bufferizado: reduce el ruido, ideal para salidas de audio.
- Trigger externo: puede sincronizarse con un temporizador (por ejemplo TIM6/TRGO).
- Soporte DMA: se permite la alimentación directa de datos desde memoria al DAC sin CPU.
- Salidas disponibles: a través de pines analógicos dedicados (por ejemplo, PA4 para DAC1).

3.3.3. DMA (Direct Memory Access)

El controlador DMA2 es utilizado para las transferencias entre ADC/DAC y RAM. El STM32F446RE cuenta con dos controladores DMA (DMA1 y DMA2), con un total de 16 canales (streams) y soporte para múltiples periféricos. Características específicas:

- Transferencia en background, sin intervención del CPU.
- Modo circular: ideal para buffers cíclicos o doble buffering.
- Prioridad programable por canal.
- Transferencia por palabras (16 bits en Q15).
- Eventos por mitad de buffer y buffer completo: permite usar doble buffer sincronizado.

3.3.4. TIM (Timers)

El STM32F446RE cuenta con 14 temporizadores (4 avanzados, 2 básicos, 8 generales), todos con configuraciones de prescaler, auto-reload y posibilidad de generar eventos de actualización. Se utilizan para generar señales periódicas que actúan como trigger de muestreo. Características clave:

- TIM2-TIM5: de 32 bits, con amplio rango de conteo (útiles para precisión en frecuencias bajas).
- Configuración de frecuencia: vía Prescaler y ARR (Auto-Reload Register).
- Trigger de ADC/DAC: mediante señal TRGO (Trigger Output).
- Modo master-slave: permite sincronización entre periféricos.

3.3.5. GPIO – General Purpose Input/Output

Los pines GPIO permiten la interacción con el usuario mediante botones (teclas) y LEDs. Se utilizan:

- Velocidad de salida seleccionable: Low, Medium, Fast, High.
- Pull-up/Pull-down configurables.
- Interrupciones por flanco ascendente, descendente o ambos.

3.3.6. Herramientas de desarrollo

Para la implementación del proyecto se utilizaron las siguientes herramientas:

- STM32CubeIDE: entorno de desarrollo oficial de STMicroelectronics, basado en Eclipse, que permite configurar periféricos gráficamente, generar código y compilar con GCC.
- STM32CubeMX: herramienta para la configuración inicial de pines, relojes, periféricos y generación de código base.
- CMSIS y CMSIS-DSP: bibliotecas proporcionadas por ARM para facilitar el uso de instrucciones de bajo nivel y rutinas DSP optimizadas en punto fijo.
- GDB y ST-Link: utilizados para depuración, profiling y medición de tiempos mediante DWT->CYCCNT.

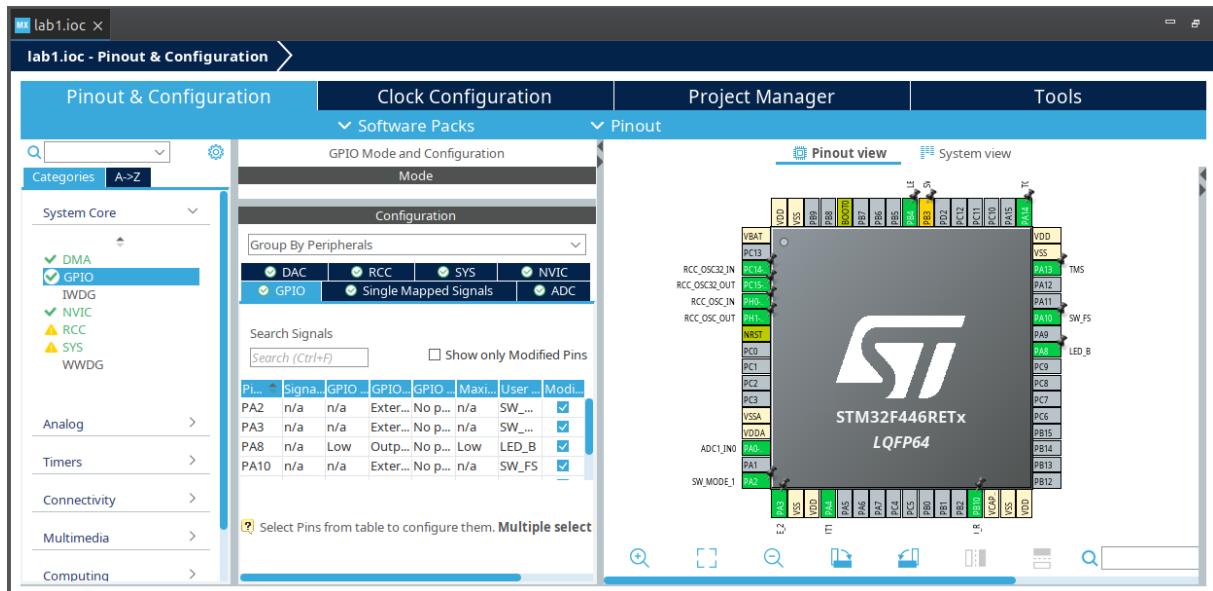


Figura 2: Entorno de desarrollo STM32CUBE IDE

4. Diseño del sistema

4.1. Descripción detallada del funcionamiento

El flujo de datos comienza con la conversión analógica-digital realizada por el ADC1, configurado en modo de escaneo continuo y disparado por el temporizador TIM2. El resultado de estas conversiones es almacenado directamente en el buffer `adc_buffer` mediante DMA en modo circular, lo cual permite una operación ininterrumpida sin intervención del procesador. El buffer es de tipo doble (double-buffered), con interrupciones generadas al completar la mitad y el total del buffer, utilizando los callbacks `HAL_ADC_ConvHalfCpltCallback` y `HAL_ADC_ConvCpltCallback`, respectivamente.

Cada vez que se activa uno de estos callbacks, se llama a la función `process_buffer_half`, la cual toma la mitad correspondiente del buffer de entrada, realiza la conversión a formato Q15 (`q15_t`) y, opcionalmente, podría aplicar un procesamiento digital (aunque en este código no se incluye ningún filtro). Posteriormente, los datos en Q15 son convertidos al formato requerido por el DAC de 12 bits mediante la función `convert_q15_to_dac_buffer`, la cual realiza un escalado proporcional y recorte para mantener los valores en el rango `[0, 4095]`. El resultado se almacena en `dac_buffer` y se reproduce a través del canal 1 del DAC, también utilizando DMA.

Un aspecto destacable del sistema es el uso del contador de ciclos del DWT (Data Watchpoint and Trace) para medir el tiempo exacto de procesamiento de cada mitad del buffer. Este contador se activa en el `main` y se lee al principio y final de cada ejecución de `process_buffer_half`, almacenando la diferencia en la variable global `time`. Esta métrica resulta útil para analizar el rendimiento del sistema o para evaluar el impacto del procesamiento DSP si se agregan etapas de filtrado más adelante.

El sistema también incluye un mecanismo de selección de frecuencia de muestreo a través de una interrupción externa generada por un pulsador conectado al pin `SW_FS`. Cada vez que se detecta una pulsación, se incrementa el valor del modo de muestreo `fs_mode`, recorriendo un ciclo de seis estados. En cada modo, se actualiza el valor de auto-recarga del temporizador TIM2 (ARR) para modificar la frecuencia de disparo del ADC, alcanzando frecuencias de muestreo aproximadas de 8 kHz, 16 kHz, 24 kHz, 40 kHz

y 48 kHz. El sexto modo detiene por completo el muestreo, apagando el temporizador y apagando todos los LEDs indicadores.

Los LEDs conectados a los pines LED_B, LED_R y LED_G actúan como indicadores visuales del modo actual de frecuencia de muestreo. Esto permite al usuario reconocer rápidamente el estado del sistema sin requerir una interfaz adicional.

La función **main** inicializa los periféricos esenciales mediante llamadas generadas por CubeMX: ADC, DAC, DMA, GPIO y el temporizador TIM2. Luego activa el sistema de temporización, inicia la conversión del ADC en modo DMA, así como la salida del DAC también por DMA, y deja el bucle principal completamente vacío. Todo el flujo de datos y procesamiento se lleva a cabo de manera asíncrona mediante interrupciones y DMA, garantizando una alta eficiencia y baja latencia.

Este enfoque modular, escalable y eficiente lo convierte en una base robusta para sistemas de procesamiento de señal embebido, como filtros en tiempo real, analizadores de espectro o codificadores de audio, con la posibilidad de extender fácilmente su funcionalidad agregando etapas DSP en el dominio Q15 aprovechando la biblioteca CMSIS-DSP.

4.2. Diagrama de bloques del programa aplicativo

El siguiente diagrama de bloques resume el flujo funcional del sistema de adquisición y reproducción de señales en tiempo real.

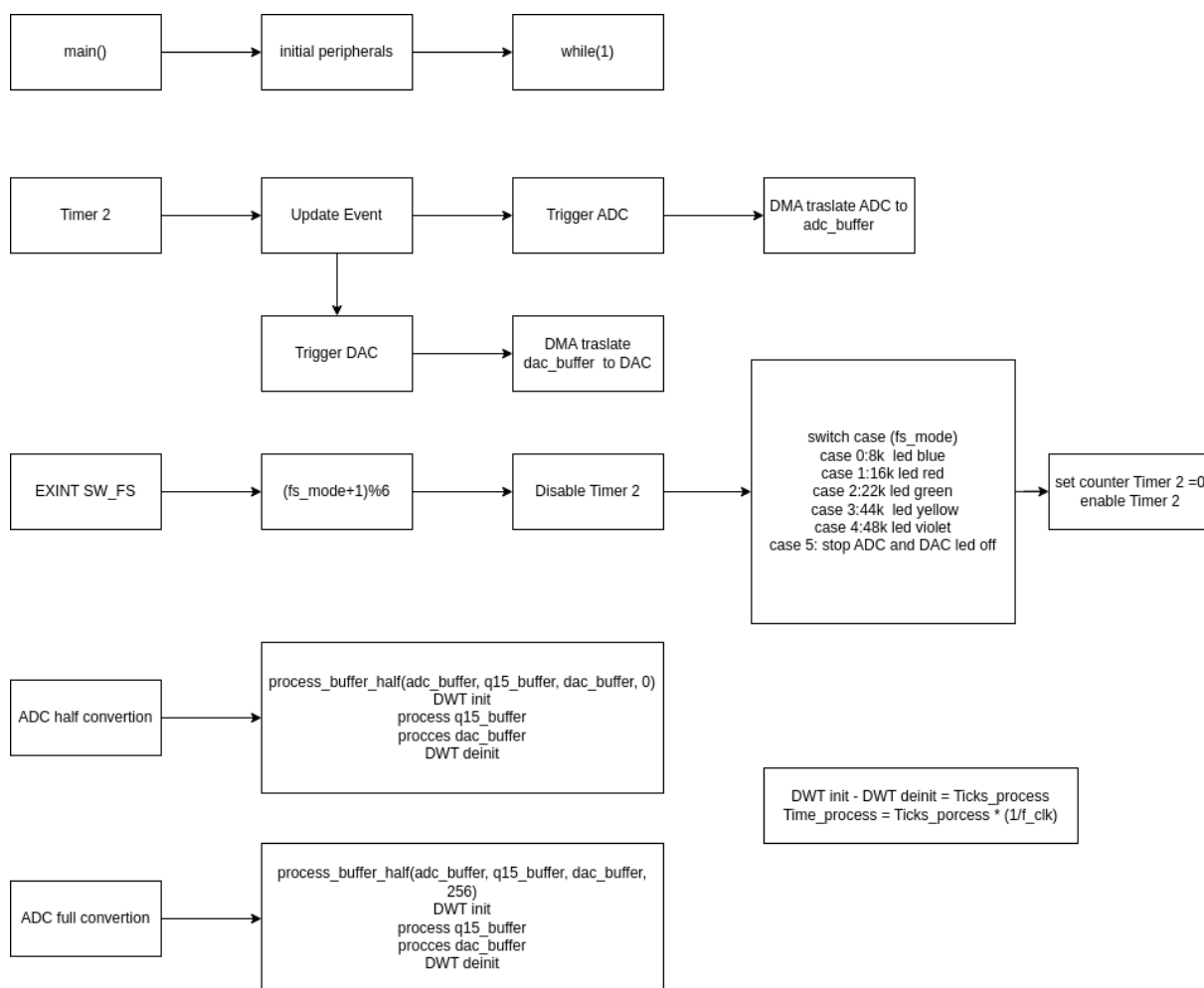


Figura 3: Diagrama de flujo TOP LEVEL

5. Programa principal

5.1. Implementación del main.c

```

1  /**
2  ****
3  * @file           : main.c
4  * @brief          : Real-Time ADC Sampling and Playback via DAC using DMA
5  *
6  * This program performs real-time signal acquisition and playback on the
7  *   STM32F469RE
8  * microcontroller. It continuously samples an analog input signal using the ADC,
9  *   converts the samples to Q15 fixed-point format, and reproduces them via the DAC
10 *
11 * The entire process is handled using DMA and operates in a double-buffered
12 *   fashion
13 * to ensure minimal CPU usage. The system includes a performance profiling
14 *   mechanism
15 * based on the DWT cycle counter.
16 *
17 * Key Features:
18 * - 12-bit ADC sampling with DMA in circular mode
19 * - Q15 fixed-point format conversion
20 * - 12-bit DAC output via DMA
21 * - Processing time measurement using DWT->CYCCNT
22 * - Configurable sampling frequency via TIM2
23 *
24 * Author           : Campos Mariano
25 * Date             : 25/06/2025
26 ****
27 */
28
29 /* USER CODE END Header */
30 /* Includes -----*/
31 #include "main.h"
32 #include "adc.h"
33 #include "dac.h"
34 #include "dma.h"
35 #include "tim.h"
36 #include "gpio.h"
37
38 /* Private includes -----*/
39 /* USER CODE BEGIN Includes */
40 #include "arm_math.h" // CMSIS-DSP library for fixed-point processing
41 /* USER CODE END Includes */
42
43 /* Private typedef -----*/
44 /* USER CODE BEGIN PTD */
45
46 /* USER CODE END PTD */
47
48 /* Private define -----*/
49 /* USER CODE BEGIN PD */
50 #define BUFFER_SIZE 512 // Size of ADC/DAC/Q15 buffers
51 #define FS_8K 11249 // Auto-reload value for TIM2 to achieve ~8kHz
52 // sampling with 90 MHz clock
53 /* USER CODE END PD */
54
55 /* Private variables -----*/
56
57 /* USER CODE BEGIN PV */
58 extern TIM_HandleTypeDef htim2; // Timer used for triggering ADC/DAC
59 extern ADC_HandleTypeDef hadc1; // ADC handle
60 extern DAC_HandleTypeDef hdac; // DAC handle
61
62 uint16_t adc_buffer[BUFFER_SIZE]; // Raw ADC samples
63 uint16_t dac_buffer[BUFFER_SIZE]; // Output buffer for DAC
64 q15_t q15_buffer[BUFFER_SIZE]; // Intermediate Q15 format buffer
65 uint8_t fs_mode = 0; // Sampling frequency mode selector
66 uint32_t time = 0; // Variable to store CPU cycles for
67 // processing time profiling
68 /* USER CODE END PV */
69
70 /* Private user code -----*/

```

```

65  /* USER CODE BEGIN 0 */
66
67  /**
68   * @brief Converts a Q15 array to a 12-bit DAC buffer
69   * @param in: Input Q15 data array
70   * @param out: Output DAC buffer
71   * @param len: Number of elements to convert
72   */
73  void convert_q15_to_dac_buffer(q15_t *in, uint16_t *out, uint16_t len) {
74      for (uint16_t i = 0; i < len; i++) {
75          int32_t val = ((int32_t) in[i] * 4095) / 32767;
76          if (val < 0)
77              val = 0;
78          else if (val > 4095)
79              val = 4095;
80          out[i] = (uint16_t) (val);
81      }
82  }
83
84  /**
85   * @brief Processes half of the ADC buffer: converts to Q15, applies processing,
86   *        and prepares DAC output
87   * @param adc_in: Input ADC buffer
88   * @param q15_out: Intermediate Q15 buffer
89   * @param dac_out: Output DAC buffer
90   * @param offset: Offset to select first or second half of buffer
91   */
92  void process_buffer_half(uint16_t *adc_in, q15_t *q15_out, uint16_t *dac_out,
93                          uint16_t offset) {
94      uint32_t start = DWT->CYCCNT; // Start cycle counter for performance profiling
95
96      // Convert ADC 12-bit values to Q15 format
97      for (uint16_t i = 0; i < BUFFER_SIZE / 2; i++) {
98          q15_out[i + offset] = (q15_t)(((int32_t) adc_in[i + offset] * 32767) / 4095);
99      }
100
101      // Convert processed Q15 data to DAC output format
102      convert_q15_to_dac_buffer(&q15_out[offset], &dac_out[offset], BUFFER_SIZE / 2);
103      time = DWT->CYCCNT - start; // Measure processing time in CPU cycles
104  }
105
106  /**
107   * @brief DMA Half Transfer Complete Callback for ADC
108   */
109  void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc) {
110      if (hadc->Instance == ADC1) {
111          process_buffer_half(adc_buffer, q15_buffer, dac_buffer, 0);
112      }
113  }
114
115  /**
116   * @brief DMA Transfer Complete Callback for ADC
117   */
118  void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
119      if (hadc->Instance == ADC1) {
120          process_buffer_half(adc_buffer, q15_buffer, dac_buffer, BUFFER_SIZE / 2);
121      }
122  }
123
124  /**
125   * @brief GPIO interrupt callback for switching sampling frequencies or stopping
126   *        acquisition
127   *        Cycles through 6 modes on button press (pin: SW_FS)
128   */
129  void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
130      if (GPIO_Pin == SW_FS_Pin) {
131          fs_mode = (fs_mode + 1) % 6;
132
133          // Stop timer before reconfiguring
134          __HAL_TIM_DISABLE(&htim2);
135
136          switch (fs_mode) {
137              case 0:
138                  __HAL_TIM_SET_AUTORELOAD(&htim2, FS_8K); // 8 kHz

```

```

138     HAL_GPIO_WritePin(LED_B_GPIO_Port , LED_B_Pin , GPIO_PIN_SET);           // Blue
139     HAL_GPIO_WritePin(LED_R_GPIO_Port , LED_R_Pin , GPIO_PIN_RESET);
140     HAL_GPIO_WritePin(LED_G_GPIO_Port , LED_G_Pin , GPIO_PIN_RESET);
141     break;
142     case 1:
143         __HAL_TIM_SET_AUTORELOAD(&htim2, FS_8K / 2);           // ~16 kHz
144         HAL_GPIO_WritePin(LED_B_GPIO_Port , LED_B_Pin , GPIO_PIN_RESET);
145         HAL_GPIO_WritePin(LED_R_GPIO_Port , LED_R_Pin , GPIO_PIN_SET);           // Red
146         HAL_GPIO_WritePin(LED_G_GPIO_Port , LED_G_Pin , GPIO_PIN_RESET);
147         break;
148     case 2:
149         __HAL_TIM_SET_AUTORELOAD(&htim2, FS_8K / 3);           // ~22 kHz
150         HAL_GPIO_WritePin(LED_B_GPIO_Port , LED_B_Pin , GPIO_PIN_RESET);
151         HAL_GPIO_WritePin(LED_R_GPIO_Port , LED_R_Pin , GPIO_PIN_RESET);
152         HAL_GPIO_WritePin(LED_G_GPIO_Port , LED_G_Pin , GPIO_PIN_SET);           // Green
153         break;
154     case 3:
155         __HAL_TIM_SET_AUTORELOAD(&htim2, FS_8K / 5);           // ~44 kHz
156         HAL_GPIO_WritePin(LED_B_GPIO_Port , LED_B_Pin , GPIO_PIN_RESET);
157         HAL_GPIO_WritePin(LED_R_GPIO_Port , LED_R_Pin , GPIO_PIN_SET);           // Red +
158             Green = Yellow
159         HAL_GPIO_WritePin(LED_G_GPIO_Port , LED_G_Pin , GPIO_PIN_SET);
160         break;
161     case 4:
162         __HAL_TIM_SET_AUTORELOAD(&htim2, FS_8K / 6);           // ~48 kHz
163         HAL_GPIO_WritePin(LED_B_GPIO_Port , LED_B_Pin , GPIO_PIN_SET);           // Blue +
164             Red = Purple
165         HAL_GPIO_WritePin(LED_R_GPIO_Port , LED_R_Pin , GPIO_PIN_SET);
166         HAL_GPIO_WritePin(LED_G_GPIO_Port , LED_G_Pin , GPIO_PIN_RESET);
167         break;
168     case 5:
169         // Stop all activity
170         HAL_GPIO_WritePin(LED_B_GPIO_Port , LED_B_Pin , GPIO_PIN_RESET);           // All
171             off
172         HAL_GPIO_WritePin(LED_R_GPIO_Port , LED_R_Pin , GPIO_PIN_RESET);
173         HAL_GPIO_WritePin(LED_G_GPIO_Port , LED_G_Pin , GPIO_PIN_RESET);
174         __HAL_TIM_DISABLE(&htim2);
175         return;
176     }
177
178     // Reset and enable timer
179     __HAL_TIM_SET_COUNTER(&htim2, 0);
180     TIM2->EGR = TIM_EGR_UG;
181     __HAL_TIM_ENABLE(&htim2);
182 }
183
184 /* USER CODE END 0 */
185
186 /**
187  * @brief The application entry point.
188  * @retval int
189  */
190 int main(void)
191 {
192     /* USER CODE BEGIN 1 */
193     // Enable DWT cycle counter for profiling purposes
194     CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
195     DWT->CYCCNT = 0;
196     DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
197     /* USER CODE END 1 */
198
199     /* MCU Configuration ----- */
200
201     /* Reset of all peripherals, Initializes the Flash interface and the Systick.
202      */
203     HAL_Init();
204
205     /* USER CODE BEGIN Init */
206
207     /* USER CODE END Init */
208
209     /* Configure the system clock */
210     SystemClock_Config();
211
212     /* USER CODE BEGIN SysInit */

```

```

210
211     /* USER CODE END SysInit */
212
213     /* Initialize all configured peripherals */
214     MX_GPIO_Init();
215     MX_DMA_Init();
216     MX_ADC1_Init();
217     MX_TIM2_Init();
218     MX_DAC_Init();
219     /* USER CODE BEGIN 2 */
220
221     HAL_GPIO_WritePin(LED_B_GPIO_Port, LED_B_Pin, GPIO_PIN_SET);    // Blue
222     HAL_GPIO_WritePin(LED_R_GPIO_Port, LED_R_Pin, GPIO_PIN_RESET);
223     HAL_GPIO_WritePin(LED_G_GPIO_Port, LED_G_Pin, GPIO_PIN_RESET);
224
225     // Start base timer
226     HAL_TIM_Base_Start(&htim2);
227
228     // Start DAC in DMA mode
229     HAL_DAC_Start_DMA(&hdac, DAC_CHANNEL_1, (uint32_t*) dac_buffer, BUFFER_SIZE,
        DAC_ALIGN_12B_R);
230
231     // Start ADC in DMA mode
232     HAL_ADC_Start_DMA(&hadc1, (uint32_t*) adc_buffer, BUFFER_SIZE);
233
234     /* USER CODE END 2 */
235
236     /* Infinite loop */
237     /* USER CODE BEGIN WHILE */
238     while (1) {
239         /* USER CODE END WHILE */
240
241         /* USER CODE BEGIN 3 */
242         // Main loop intentionally left empty. All processing handled via DMA
        // callbacks and interrupts.
243     }
244     /* USER CODE END 3 */
245 }
246
247 /**
248  * @brief System Clock Configuration
249  * @retval None
250  */
251 void SystemClock_Config(void)
252 {
253     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
254     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
255
256     /** Configure the main internal regulator output voltage
257     */
258     __HAL_RCC_PWR_CLK_ENABLE();
259     __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
260
261     /** Initializes the RCC Oscillators according to the specified parameters
262     * in the RCC_OscInitTypeDef structure.
263     */
264     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
265     RCC_OscInitStruct.HSISState = RCC_HSI_ON;
266     RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
267     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
268     RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
269     RCC_OscInitStruct.PLL.PLLM = 8;
270     RCC_OscInitStruct.PLL.PLLN = 180;
271     RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
272     RCC_OscInitStruct.PLL.PLLQ = 2;
273     RCC_OscInitStruct.PLL.PLLR = 2;
274     if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
275     {
276         Error_Handler();
277     }
278
279     /** Activate the Over-Drive mode
280     */
281     if (HAL_PWREx_EnableOverDrive() != HAL_OK)
282     {
283         Error_Handler();

```

```

284     }
285
286     /** Initializes the CPU, AHB and APB buses clocks */
287     /*
288     RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYCLK
289     |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
290     RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
291     RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
292     RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
293     RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
294
295     if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5) != HAL_OK)
296     {
297         Error_Handler();
298     }
299 }
300
301 /**
302 * @brief This function is executed in case of error occurrence.
303 * @retval None
304 */
305 void Error_Handler(void)
306 {
307     /* USER CODE BEGIN Error_Handler_Debug */
308     // Stay in infinite loop on error
309     __disable_irq();
310     while (1) {
311     }
312     /* USER CODE END Error_Handler_Debug */
313 }
314
315 #ifdef USE_FULL_ASSERT
316 /**
317 * @brief Reports the name of the source file and the source line number
318 * where the assert_param error has occurred.
319 * @param file: pointer to the source file name
320 * @param line: assert_param error line source number
321 * @retval None
322 */
323 void assert_failed(uint8_t *file, uint32_t line)
324 {
325     /* USER CODE BEGIN 6 */
326     // Custom assert handler (optional user-defined implementation)
327     /* USER CODE END 6 */
328 }
329 #endif /* USE_FULL_ASSERT */

```

5.2. Configuración inicial de los periféricos

5.2.1. Configuración del ADC

```

1 /* ADC1 init function */
2 void MX_ADC1_Init(void){
3
4     ADC_ChannelConfTypeDef sConfig = {0};
5
6     /** Configure the global features of the ADC (Clock, Resolution, Data Alignment
7     and number of conversion)
8     */
9     hadc1.Instance = ADC1;
10    hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV4;
11    hadc1.Init.Resolution = ADC_RESOLUTION_12B;
12    hadc1.Init.ScanConvMode = DISABLE;
13    hadc1.Init.ContinuousConvMode = DISABLE;
14    hadc1.Init.DiscontinuousConvMode = DISABLE;
15    hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_RISING;
16    hadc1.Init.ExternalTrigConv = ADC_EXTERNALTRIGCONV_T2_TRGO;
17    hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
18    hadc1.Init.NbrOfConversion = 1;
19    hadc1.Init.DMAContinuousRequests = ENABLE;
20    hadc1.Init.EOCSelection = ADC_EOC_SINGLE_CONV;
21    if (HAL_ADC_Init(&hadc1) != HAL_OK)

```



```

21     {
22         Error_Handler();
23     }
24
25     /** Configure for the selected ADC regular channel its corresponding rank in
26     the sequencer and its sample time.
27     */
28     sConfig.Channel = ADC_CHANNEL_0;
29     sConfig.Rank = 1;
30     sConfig.SamplingTime = ADC_SAMPLETIME_3CYCLES;
31     if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
32     {
33         Error_Handler();
34     }

```

5.2.2. Configuración del DAC

```

1  /* DAC init function */
2  void MX_DAC_Init(void){
3
4      DAC_ChannelConfTypeDef sConfig = {0};
5
6      /** DAC Initialization
7      */
8      hdac.Instance = DAC;
9      if (HAL_DAC_Init(&hdac) != HAL_OK)
10     {
11         Error_Handler();
12     }
13     /** DAC channel OUT1 config
14     */
15     sConfig.DAC_Trigger = DAC_TRIGGER_T2_TRGO;
16     sConfig.DAC_OutputBuffer = DAC_OUTPUTBUFFER_ENABLE;
17     if (HAL_DAC_ConfigChannel(&hdac, &sConfig, DAC_CHANNEL_1) != HAL_OK)
18     {
19         Error_Handler();
20     }
21 }

```

5.2.3. Configuración del GPIO

```

1
2  /** Configure pins as
3  * Analog
4  * Input
5  * Output
6  * EVENT_OUT
7  * EXTI
8  */
9  void MX_GPIO_Init(void)
10 {
11
12     GPIO_InitTypeDef GPIO_InitStruct = {0};
13
14     /* GPIO Ports Clock Enable */
15     __HAL_RCC_GPIOC_CLK_ENABLE();
16     __HAL_RCC_GPIOH_CLK_ENABLE();
17     __HAL_RCC_GPIOA_CLK_ENABLE();
18     __HAL_RCC_GPIOB_CLK_ENABLE();
19
20     /*Configure GPIO pin Output Level */
21     HAL_GPIO_WritePin(GPIOB, LED_R_Pin|LED_G_Pin, GPIO_PIN_RESET);
22
23     /*Configure GPIO pin Output Level */
24     HAL_GPIO_WritePin(LED_B_GPIO_Port, LED_B_Pin, GPIO_PIN_RESET);
25
26     /*Configure GPIO pins : SW_MODE_1_Pin SW_MODE_2_Pin SW_FS_Pin */
27     GPIO_InitStruct.Pin = SW_MODE_1_Pin|SW_MODE_2_Pin|SW_FS_Pin;

```

```

28     GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
29     GPIO_InitStruct.Pull = GPIO_NOPULL;
30     HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
31
32     /*Configure GPIO pins : LED_R_Pin LED_G_Pin */
33     GPIO_InitStruct.Pin = LED_R_Pin|LED_G_Pin;
34     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
35     GPIO_InitStruct.Pull = GPIO_NOPULL;
36     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
37     HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
38
39     /*Configure GPIO pin : LED_B_Pin */
40     GPIO_InitStruct.Pin = LED_B_Pin;
41     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
42     GPIO_InitStruct.Pull = GPIO_NOPULL;
43     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
44     HAL_GPIO_Init(LED_B_GPIO_Port, &GPIO_InitStruct);
45
46     /* EXTI interrupt init*/
47     HAL_NVIC_SetPriority(EXTI2_IRQn, 0, 0);
48     HAL_NVIC_EnableIRQ(EXTI2_IRQn);
49
50     HAL_NVIC_SetPriority(EXTI3_IRQn, 0, 0);
51     HAL_NVIC_EnableIRQ(EXTI3_IRQn);
52
53     HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0, 0);
54     HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
55
56 }

```

5.2.4. Configuración del DMA

```

1  void MX_DMA_Init(void)
2  {
3
4      /* DMA controller clock enable */
5      __HAL_RCC_DMA2_CLK_ENABLE();
6      __HAL_RCC_DMA1_CLK_ENABLE();
7
8      /* DMA interrupt init */
9      /* DMA1_Stream5_IRQn interrupt configuration */
10     HAL_NVIC_SetPriority(DMA1_Stream5_IRQn, 0, 0);
11     HAL_NVIC_EnableIRQ(DMA1_Stream5_IRQn);
12     /* DMA2_Stream0_IRQn interrupt configuration */
13     HAL_NVIC_SetPriority(DMA2_Stream0_IRQn, 0, 0);
14     HAL_NVIC_EnableIRQ(DMA2_Stream0_IRQn);
15
16 }

```

5.2.5. Configuración del TIM2

```

1  /* TIM2 init function */
2  void MX_TIM2_Init(void) {
3
4      TIM_ClockConfigTypeDef sClockSourceConfig = {0};
5      TIM_MasterConfigTypeDef sMasterConfig = {0};
6
7      /* USER CODE BEGIN TIM2_Init 1 */
8
9      /* USER CODE END TIM2_Init 1 */
10     htim2.Instance = TIM2;
11     htim2.Init.Prescaler = 0;
12     htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
13     htim2.Init.Period = 11249;
14     htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
15     htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
16     if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
17     {
18         Error_Handler();

```

```

19     }
20     sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
21     if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK)
22     {
23         Error_Handler();
24     }
25     sMasterConfig.MasterOutputTrigger = TIM_TRGO_UPDATE;
26     sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
27     if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
28     {
29         Error_Handler();
30     }
31     /* USER CODE BEGIN TIM2_Init 2 */
32     /* USER CODE END TIM2_Init 2 */
33 }

```

6. Resultados experimentales

6.1. Pruebas realizadas con diferentes velocidades de muestreo

Para evaluar la integridad de una señal muestreada se pueden realizar diversas mediciones utilizando un osciloscopio digital. En primer lugar, es fundamental verificar que la frecuencia de la señal de salida coincida con la esperada, lo cual puede comprobarse mediante la medición del período o utilizando las herramientas automáticas de medición de frecuencia que ofrece el osciloscopio. Luego, se analiza la amplitud pico a pico (V_{pp}) para comprobar que el nivel de la señal se mantiene dentro del valor nominal, sin saturaciones ni pérdidas de resolución.

Otro aspecto importante es el análisis de distorsión armónica, que se puede realizar activando la función de transformada rápida de Fourier (FFT) en el osciloscopio. Esta herramienta permite identificar la magnitud de los armónicos presentes en la señal. A partir de esta información, se puede calcular la distorsión armónica total (THD, por sus siglas en inglés) mediante la fórmula:

$$THD = \frac{\sqrt{V_2^2 + V_3^2 + \dots + V_n^2}}{V_1^2} \quad (2)$$

Donde V_1 es la amplitud del primer armónico (la componente fundamental), V_2 , V_3 corresponden a las amplitudes de los armónicos superiores. Además, se puede estimar la relación señal a ruido (SNR) observando la diferencia en dB entre el nivel de la fundamental y el nivel del ruido de fondo en el espectro. También es recomendable inspeccionar visualmente la forma de onda en el dominio temporal para detectar posibles errores de reconstrucción, tales como escalones, aliasing, jitter o glitches, los cuales son indicativos de una mala conversión o de una tasa de muestreo insuficiente.

Para realizar las medidas se utiliza una frecuencia monotonó senoidal del 4 kHz, con una tensión pico a pico de 3.3V, sin cambio de polaridad. Esto con la finalidad de pueda ser muestreada al menos con dos muestras por ciclo, y que sea correctamente digitalizada por un ADC con una tensión de referencia de 0V a 3.3V. Las muestras medidas por el osciloscopio se guardan y se muestran a continuación:

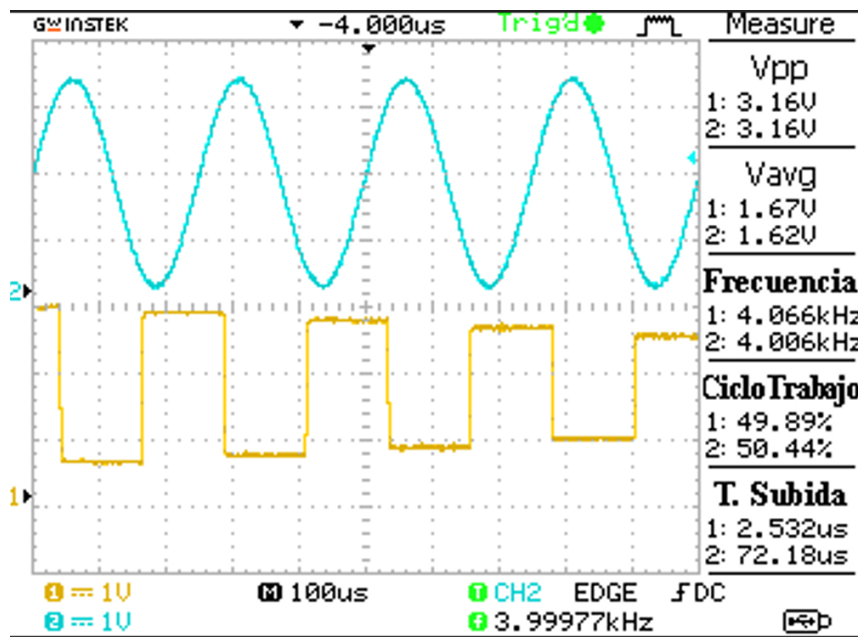


Figura 4: Medición de la señal a frecuencia de muestreo de 8 kHz

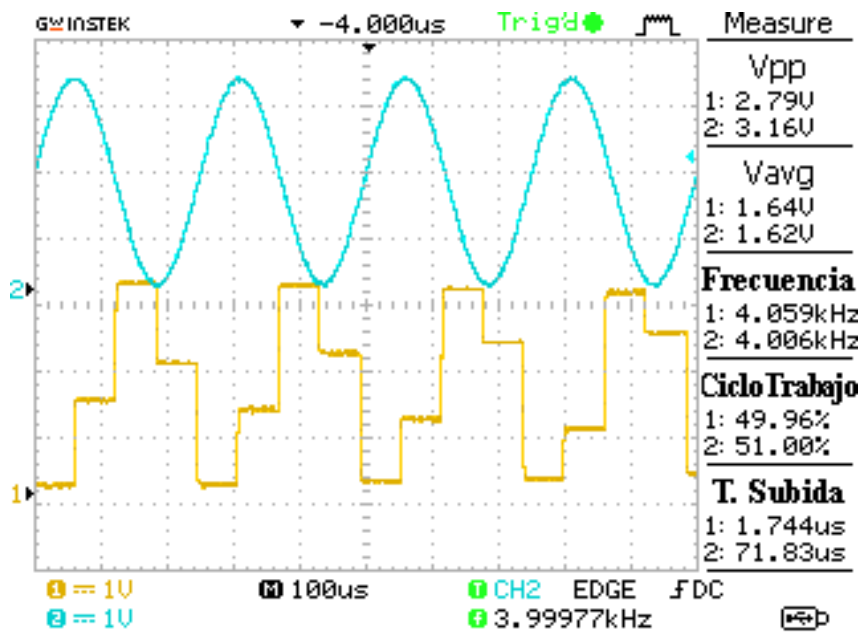


Figura 5: Medición de la señal a frecuencia de muestreo de 16 kHz

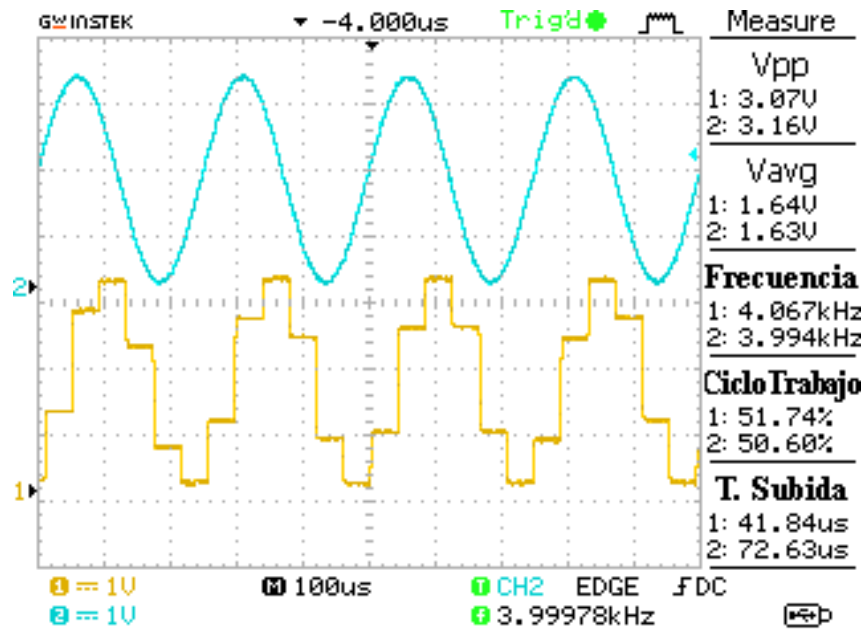


Figura 6: Medición de la señal a frecuencia de muestreo de 24 kHz

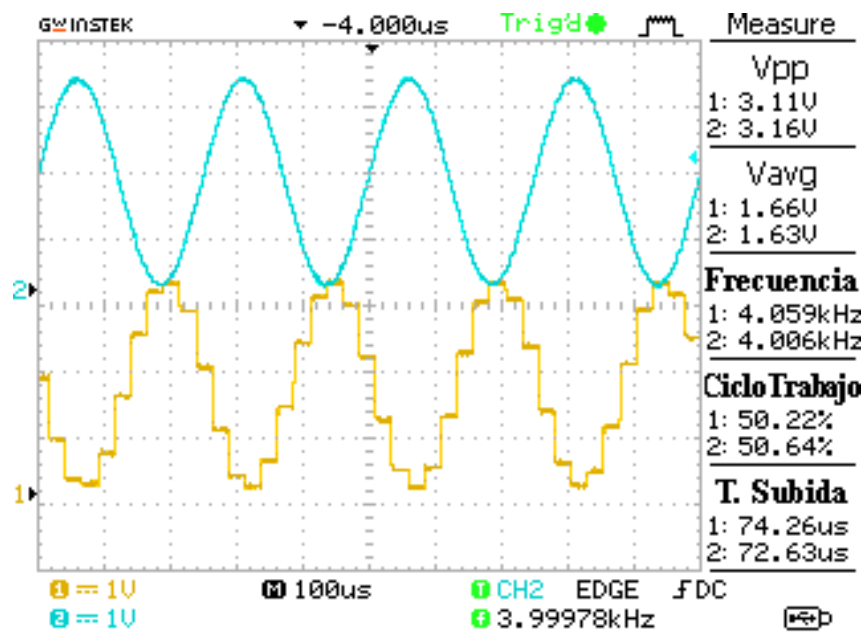


Figura 7: Medición de la señal a frecuencia de muestreo de 40 kHz

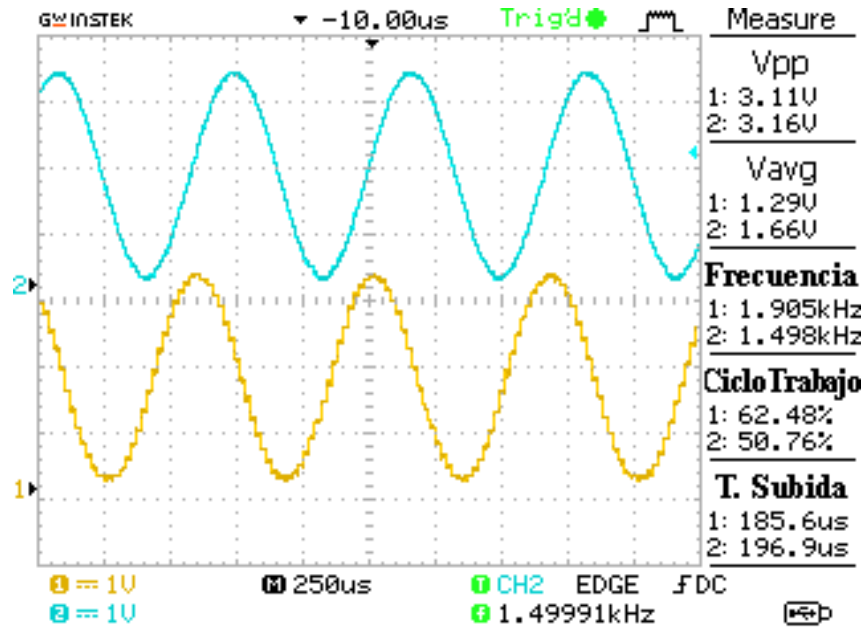


Figura 8: Medición de la señal a frecuencia de muestreo de 48 kHz

6.2. Análisis de los resultados obtenidos

f_s [kHz]	Muestras por ciclo (N)	V_{pp} [mV]	V_{rms} [mV]	THD [%]	SNR [dB]
8	2	3360	2047.8	6.02	35.00
16	4	3280	1931.5	8.69	35.59
24	6	3280	1917.8	1.80	36.17
40	10	3200	1943.1	3.58	36.12
48	12	3280	1925.1	4.20	36.31

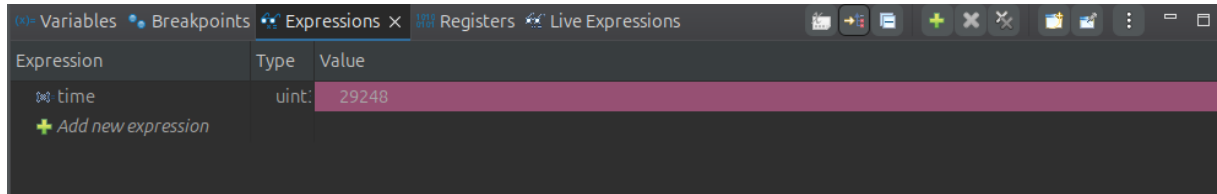
Cuadro 1: Parámetros de la señal de prueba: $f_s = 4[kHz]$, $V_{pp} = 3,3[V]$

7. Conclusiones

La señal de prueba fue generada con un modulo AD9833, la resolución de salida es mediante un DAC de 10 bits, ademas de presentar jitter y variaciones en la amplitud pico, por lo que se podría obtener resultados mejores con un generador de funciones con mas resolución, respecto a los parámetros de las señales en todos los casos se obtiene una SNR menor a los -35 dB. Para la distorsión armónica se obtienen valores aceptables para una velocidad de muestreo mayor a las 6 muestras por ciclo. No presenta fenómenos de aliasing ni glitches para ninguna velocidad de muestreo prevista. Ademas se midió la cantidad de ciclos por instrucciones necesarios para el procesamiento de señales arrojando el siguiente resultado:

Si tenemos en cuenta que el microcontrolador trabaja a una velocidad de reloj de 180 MHz, podemos realizar los siguientes cálculos:

$$f_{CPU} = 180[MHz] \quad (3)$$



Expression	Type	Value
time	uint	29248
+ Add new expression		

Figura 9: Resultados modulo DWT

$$T_{CPU} = \frac{1}{f_{CPU}} = 5,56[nS] \quad (4)$$

$$n_{Cycles} = 29248 \quad (5)$$

Por lo tanto el periodo de procesamiento es de:

$$T_{Processing} = T_{CPU}n_{Cycles} = 162,59 [uS] \quad (6)$$

Si tomamos el caso mas desfavorable que es cuanto la velocidad de muestreo de 48 kHz, tenemos que la ventana de tiempo del microcontrolador para realizar el procesamiento de medio buffer:

$$n_{Samples} = 256 \quad (7)$$

$$T_{Sample} = \frac{1}{f_{Sample}} = 10,43[us] \quad (8)$$

$$T_{threshold} = n_{Samples}T_{Sample} = 5,33[ms] \quad (9)$$

Por lo tanto podemos concluir que el margen es de lo suficientemente grande como para realizar tanto el muestreo de la señal propiamente dicho así como también procesamiento adicional, el trabajo practico representa una solida base para los próximos trabajos de laboratorio que consiste en filtrado de señales, análisis espectrales, etc.

8. Bibliografía

Referencias

- [1] ARM Ltd., *CMSIS-DSP Software Library*, Disponible en: <https://www.keil.com/pack/doc/CMSIS/DSP/html/index.html>, [Accedido: julio 2025].
- [2] STMicroelectronics, *STM32F446xC/E Datasheet - ARM Cortex-M4 32-bit MCU+FPU, 512 KB Flash, 180 MHz, ART, 3 ADCs, 2 DACs, USB OTG*, Disponible en: <https://www.st.com/resource/en/datasheet/stm32f446re.pdf>, [Accedido: julio 2025].
- [3] STMicroelectronics, *RM0390 - Reference manual STM32F446xx advanced ARM®-based 32-bit MCUs*, Disponible en: https://www.st.com/resource/en/reference_manual/dm00135183-stm32f446xx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf, [Accedido: julio 2025].

- [4] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed., Pearson Education, 2011.
- [5] M. Kubat, *An Introduction to Machine Learning*, Springer, 2017. (Capítulo introductorio sobre DSP aplicado a sistemas embebidos).
- [6] STMicroelectronics, *STM32CubeIDE User Guide*, Disponible en: <https://www.st.com/en/development-tools/stm32cubeide.html>, [Accedido: julio 2025].
- [7] STMicroelectronics, *STM32CubeMX User Manual*, Disponible en: <https://www.st.com/en/development-tools/stm32cubemx.html>, [Accedido: julio 2025].